Thadomal Shahani Engineering College, Bandra

Name: Yash Singh	**Subject: Agile Development & DevOps**
T.E. Batch: T22 Sem: 5th Roll No: 2401118	**Date: 6th August 2026**

Department of Artificial Intelligence & Data Science

Experiment No. 4

Continuous Integration, Jenkins Installation and Building a Pipeline using Jenkins

Part A: Continuous Integration and Jenkins Installation

Aim

To understand Continuous Integration, and to install Jenkins.

LO Mapped

LO3

Theory

Continuous Integration is a development practice that promotes collaboration, early issue detection, and automated testing to enhance the quality and efficiency of software development.

Jenkins is an open-source automation server that facilitates continuous integration and continuous delivery (CI/CD) of software projects. It helps developers automate various stages of the software development lifecycle, from building and testing to deployment and monitoring. Jenkins supports a wide range of plugins and integrations, making it a versatile tool for automating various tasks in software development.

Steps Followed while Installing Jenkins

I. **Java Installation:** ensured the Java Development Kit (JDK) was installed on the system, since Jenkins requires a JVM to run.

II. **Download Jenkins:** obtained the latest Jenkins WAR file (jenkins.war) from the official Jenkins website.

III. **Launch Terminal:** opened the Command Prompt and navigated to the Downloads folder containing jenkins.war.

```
java -jar C:\Users\15L\Downloads\jenkins.war --enable-future-java
```

IV. **Start Jenkins:** ran the command above, which extracted the WAR file, started an embedded Jetty server on port 8080, and initialised the Jenkins core, plugins and job configuration.

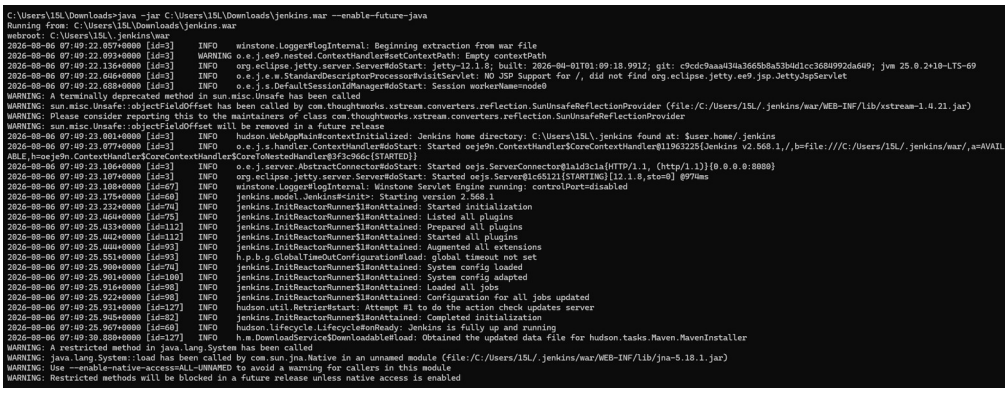


Fig. 1: Terminal output of "java -jar jenkins.war --enable-future-java", showing Jetty server startup, plugin initialisation, and the final "Jenkins is fully up and running" message

On a subsequent run, port 8080 was found to already be in use by a previous Jenkins instance. The process holding the port was identified and terminated before Jenkins was restarted successfully, as shown below.

```
C:\Users\15L\Downloads>netstat -ano | findstr :8080
  TCP    0.0.0.0:8080    0.0.0.0:0    LISTENING    5164

C:\Users\15L\Downloads>taskkill /F /PID 5164
SUCCESS: The process with PID 5164 has been terminated.

C:\Users\15L\Downloads>java -jar C:\Users\15L\Downloads\jenkins.war --enable-
future-java
```

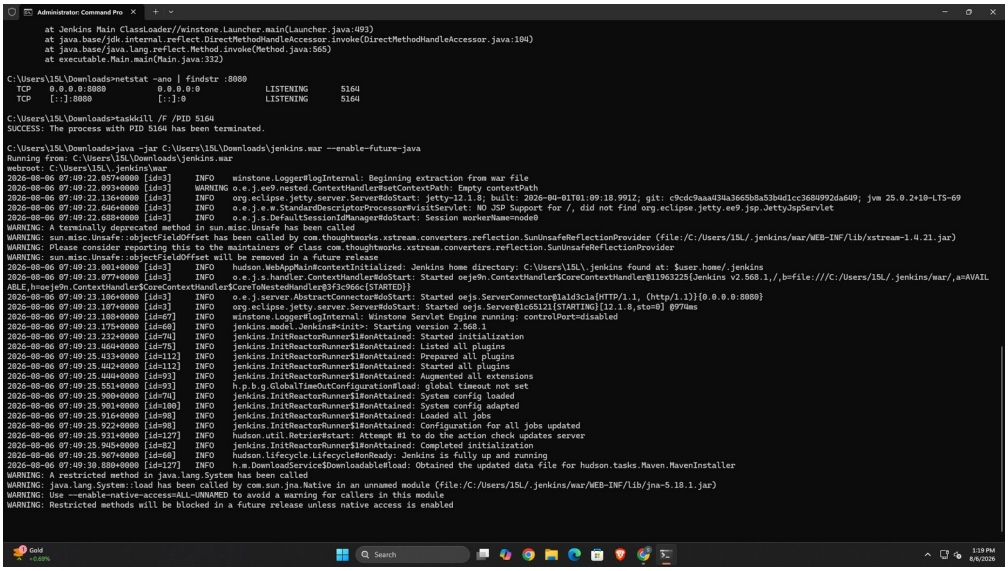


Fig. 2: Freeing port 8080 with netstat and taskkill, then restarting Jenkins successfully from the command prompt

V. Access Web UI: opened a web browser and navigated to http://localhost:8080 to reach the Jenkins setup wizard.

VI. Unlock Jenkins: retrieved the initial admin password printed in the terminal (and saved at .jenkins/secrets/initialAdminPassword) and entered it into the setup wizard.

VII. Customize Plugins: chose to install the recommended set of plugins. The Getting Started screen tracked installation progress for each plugin (Folders, OWASP Markup Formatter, Git, Pipeline, GitHub Branch Source, Timestamper, and others).

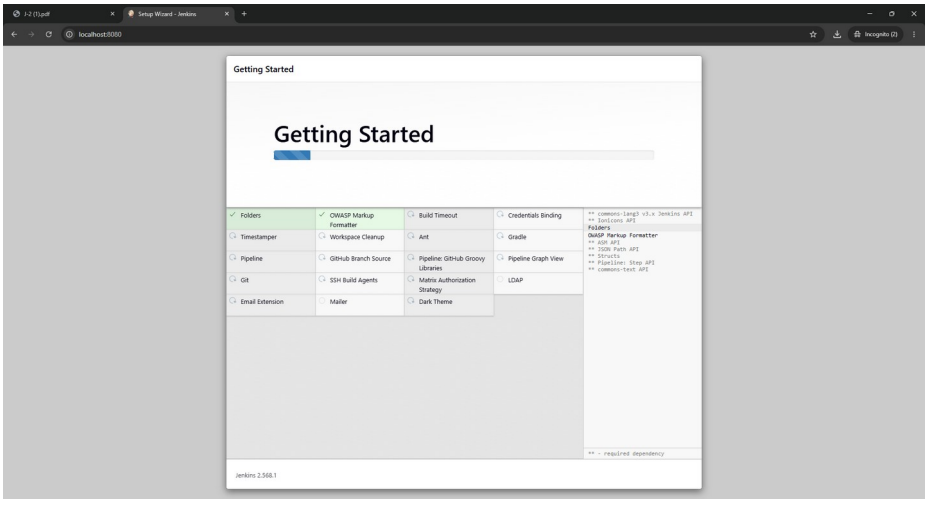


Fig. 3: Getting Started screen showing recommended plugins being installed, with a live log of dependencies on the right

VIII. Create Admin User: set up the first administrator account by entering a username, password, full name and email address.

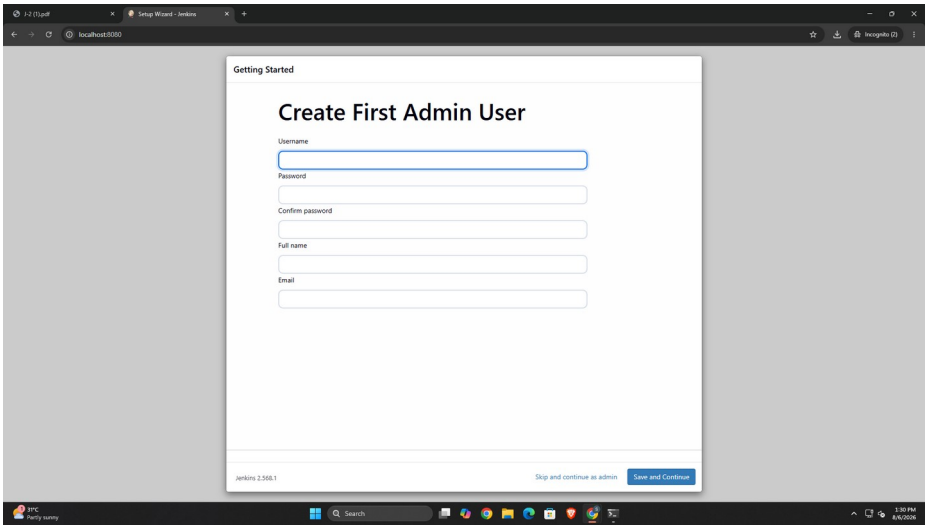


Fig. 4: "Create First Admin User" screen of the Jenkins setup wizard

IX. Jenkins URL: confirmed the default Jenkins URL (http://localhost:8080/) and completed the setup process.

X. Dashboard: logged in with the newly created admin credentials to reach the Jenkins dashboard, confirming a successful installation.

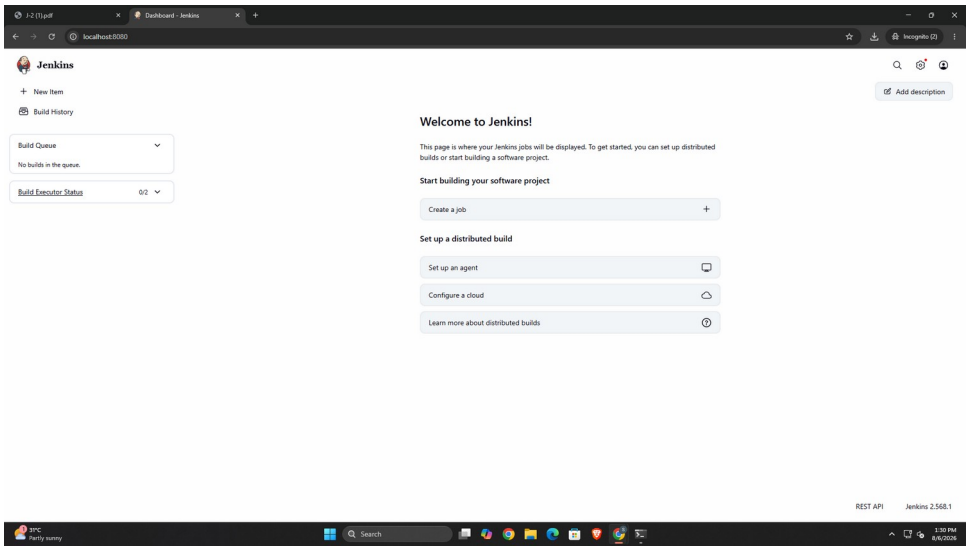


Fig. 5: Jenkins Dashboard ("Welcome to Jenkins!") after a successful installation and first login, running Jenkins version 2.568.1

Conclusion

By this assignment, Continuous Integration and the process of installing Jenkins from a WAR file — including JVM setup, first launch, port-conflict resolution, plugin installation, admin-user creation, and reaching the Jenkins dashboard — were learned and demonstrated end to end.

Part B: Building a Pipeline using Jenkins

Aim

To build a pipeline using Jenkins.

Theory

What is a Pipeline in Jenkins?

A Jenkins Pipeline is a set of automated processes that help developers and DevOps teams build, test, and deploy code in a continuous integration/continuous delivery (CI/CD) environment. In simple terms, it is a workflow that defines how software moves from code to production in an automated and repeatable manner.

Key Benefits of a Jenkins Pipeline

- Automates CI/CD processes.
- Provides better visibility into the build process.
- Offers version control of build logic (Pipeline as Code).
- Supports complex workflows such as parallel execution and conditional logic.

Types of Jenkins Pipelines

1. Declarative Pipeline — a simpler, more structured syntax designed for an easy-to-read format, introduced later to make pipelines more readable and maintainable.

- Uses a pipeline { } block.
- Organised into stages and steps.
- Easier error handling and better validation in the Jenkins UI.

2. Scripted Pipeline — a more flexible and powerful, Groovy-based syntax that gives full control over the build process and is suited to advanced users.

- Uses a node { } block.
- Everything is written as code, allowing complex scripting and logic.
- Less beginner-friendly than the declarative style.

1. Declarative Pipeline

Creation

A new Pipeline-type item (testPipeline) was created from the Jenkins dashboard. After configuration and a successful run, the job appeared in the dashboard job list along with its last build status, duration and a manual trigger (play) button.

Code

```
pipeline {
    agent any

    stages {
        stage('Hello') {
            steps {
                echo 'Hello, Jenkins!'
            }
        }
    }
}
```

Console Output

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\Users\lab1002\.jenkins\workspace\MY_TEST-1
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Hello)
[Pipeline] echo
```

```
Hello, Jenkins!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Triggering a Downstream Job

To demonstrate how one pipeline can automatically trigger another as part of a CI/CD workflow, the upstream job (yash) was configured to call a second job named default on successful completion, using the build step in its post-build action.

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Building project yash'
            }
        }
    }

    post {
        success {
            build job: 'default'
        }
    }
}
```

When the upstream job (yash) completed successfully, Jenkins automatically started build #1 of the downstream job (default). The console output of the downstream job records that it was triggered by the upstream project rather than manually, confirming the chained execution.

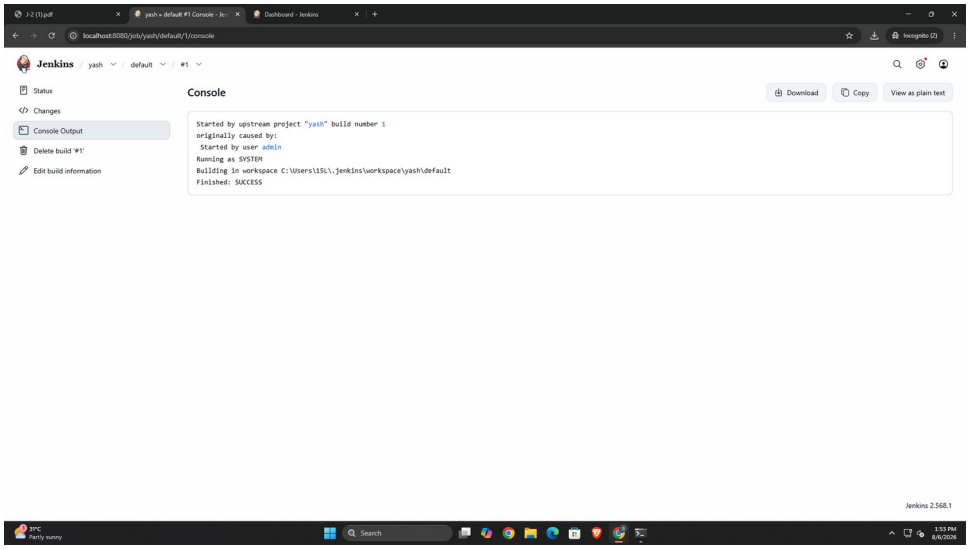


Fig. 6: Console output of the downstream job "default" (build #1), showing it was started by upstream project "yash" and finished with SUCCESS

2. Scripted Pipeline

A second item (pipeline2) was created, again selecting the Pipeline job type from Jenkins' New Item screen, this time written using the Scripted Pipeline syntax with an explicit node { } block instead of the declarative pipeline { } block, giving full programmatic control over each build step.

```
node {
    stage('Hello') {
        echo 'Hello, Jenkins! (Scripted Pipeline)'
    }
}
```

Conclusion

LO3

In this assignment, we learned how to create Jenkins Pipelines to automate the build, test, and deployment process. We explored both Declarative and Scripted pipelines, understanding their structure, benefits, and use cases, and additionally demonstrated how one pipeline can automatically trigger a downstream job on success. By implementing these pipeline examples, we demonstrated how Jenkins enables efficient, repeatable, and reliable CI/CD workflows.